

	AFSC: ANTI-FALL SMART CANE Critical Design Document	IET400_CDR_V1. 0 Revision 0.1
---	--	---



File Name: **IET400_CDR_V1.0**
Revision: 0.1
Effective
Date: 05/08/25

Universidad Popular Autónoma del Estado de Puebla
Puebla, México. 72410

Critical Design Document

Critical Design Document





AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

Owners Document	E-Mail	Position
Leader & Owner Name	santiagoalejandro.santos@upaep.edu.mx	Project Leader

Record of Revisions				
REV	DATE	AFFECTED PAGES	DESCRIPTION OF CHANGE	AUTHOR(S)
Draft	17/07/25	All	Draft Release (PDR).	All
1.0	21/07/25	All	PDR Final Release	All
2.0	23/07/25	All	CDR 1 st Review	All
3.0	25/07/25	All	CDR Final Release	All



**AFSC: ANTI FALL
SMART CANE**
Critical Design Document

**IET400_CDR_V1.0
Revision 0.1**

Table of Contents

LIST OF FIGURES.....	4
LIST OF TABLES.....	4
1.0 Document Purpose.....	5
1.1 Document Scope.....	5
1.2 Change Control and Update Procedures.....	5
2.0 Reference Documents.....	5
3.0 Concept of Operation.....	6
4.0 Project Summary.....	6
4.1 Goal.....	6
5.0 Requirements.....	7
6.0 Project Schedule.....	7
7.0 Materials, Equipment & Total Cost.....	7
7.1 Microcontroller(s).....	7
7.2 Sensors and Actuators.....	8
7.3 Other Components Required.....	8
7.4 Total Cost Of System.....	8
8.0 System Design.....	9
8.1 Structural-Mechanical System Design Diagram.....	9
8.2 Electrical System Design Diagram.....	10
8.3 Sensor System Code.....	10
9.0 Test Plan.....	10
9.1 Structural and Mechanical Testing.....	11
9.2 Electrical Function and Performance.....	11
9.3 Software Functionality Testing.....	11

10.0	Glossary.....	12
11.0	Appendices and/or Annexes.....	12

LIST OF FIGURES

- Figure 1. System Block Diagram
- Figure 2. ESP32 Wiring Diagram with Sensors
- Figure 3. Main Program Flowchart
- Figure 4. MPU6050 Fall Detection Logic
- Figure 5. MAX30102 Heart Rate Evaluation Flow
- Figure 6. PIR Inactivity Detection Flow
- Figure 7. Telegram Alert Message Example
- Figure 8. OLED Display Interface

LIST OF TABLES

- Table 1. Sensor Specifications and Characteristics
- Table 2. ESP32 GPIO Pin Assignments
- Table 3. Trigger Conditions and Thresholds
- Table 4. System State Definitions
- Table 5. WiFi and Telegram Configuration Parameters
- Table 6. Power Consumption Estimates
- Table 7. Heart Rate Interpretation Ranges
- Table 8. Component Cost Breakdown



**AFSC: ANTI FALL
SMART CANE**
Critical Design Document

**IET400_CDR_V1.0
Revision 0.1**

1.0 Document Purpose

The purpose of this document is to describe the design, development, and implementation of a smart fall detection system integrated into a walking stick. The system utilises multiple sensors to monitor potential fall events in elderly users and sends real-time alerts via Telegram. This report outlines the system architecture, sensor integration, firmware logic, testing procedures, and relevant results, demonstrating the reliability and usability of the proposed assistive device.

1.1 Document Scope

This document covers the functional and technical aspects of the Smart Cane fall detection system. It includes a detailed description of the hardware components, sensor integration, firmware structure, user interaction logic, and communication protocols. The scope is limited to the prototype development phase, focusing on fall detection, user notification through Telegram, and initial testing under controlled conditions. Future enhancements such as GPS integration, long-range communication, and improved power management are not included in this version.

1.2 Change Control and Update Procedures

Changes to this PDR document shall only be made after approval by designated representatives. Document change requests should be sent to an official team member to be submitted to a higher rank and finally to be accepted by the Project Manager.

2.0 Reference Documents

- InvenSense. (2013, agosto 19). *MPU-6000 and MPU-6050 Product Specification (Rev. 3.4)*. InvenSense.
Recuperado de <https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf>
- Adafruit Industries. (2019, noviembre 6). *MPU-6050 6-axis Accelerometer and Gyro* [Hoja técnica].
Recuperado de <https://cdn-learn.adafruit.com/downloads/pdf/mpu6050-6-axis-accelerometer-and-gyro.pdf>



**AFSC: ANTI FALL
SMART CANE**
Critical Design Document

**IET400_CDR_V1.0
Revision 0.1**

- Maxim Integrated (ahora parte de Analogue Devices). (2018, octubre). *MAX30102: High-Sensitivity Pulse Oximeter and Heart-Rate Sensor* [Datasheet, Rev. 1].
Recuperado de <https://www.analog.com/media/en/technical-documentation/data-sheets/max30102.pdf>
- DFRobot. (2021, junio 21). *Gravity: MAX30102 Heart Rate and Oximeter Sensor Module Wiki*.
Recuperado de https://wiki.dfrobot.com/SKU_SEN0518_Gravity_MAX30102_Heart_Rate_and_Oximeter_Sensor
- Joy-IT. (2017, junio 16). *KY-031 Knock-Sensor Module Datasheet*.
Recuperado de https://eclass.uth.gr/modules/document/file.php/E-CE_U_269/Sensors/Sensors_%20Datasheets/KY-031-Joy-IT.pdf

	AFSC: ANTI FALL SMART CANE Critical Design Document	IET400_CDR_V1.0 Revision 0.1
---	--	--

3.0 Concept of Operation



The intelligent cane system operates as a wearable safety device designed to monitor the user's condition and environment, providing real-time alerts in critical situations. It integrates multiple sensors—including impact detectors (KY-031), a motion sensor (MPU6050), a heart rate monitor (MAX30102), and a PIR motion detector (HW-456-SR505)—to detect falls, impacts, heart rate anomalies, and prolonged inactivity. The system processes sensor data continuously, incrementing a trigger count based on active alerts. When the trigger count reaches specific thresholds (1 for ALERT_1, 2 for ALERT_2, 3 for ALERT_3, and 4 for CRITICO), it escalates the response: from brief buzzer signals to sending Telegram notifications. In the CRITICO state (4 triggers), it sends five consecutive "Alerta crítica\nTriggers: 4" messages and activates a buzzer, giving the user 10 seconds to press a button. If pressed, it sends "Falsa alarma, todo bien" to cancel the alert; otherwise, it resets after the timeout. The OLED display provides visual feedback, showing the current state and sensor status, ensuring the user and caregivers are informed. WiFi connectivity enables remote notifications, making it a robust solution for elderly or at-risk individuals.



AFSC: ANTI FALL SMART CANE

Critical Design Document

IET400_CDR_V1.0
Revision 0.1

4.0 Project Summary

4.1 Goal

The Smart Cane is designed to ensure that elderly individuals or people with disabilities are never left alone, offering a fall detection and emergency alert system that provides continuous monitoring and support. By integrating sensors such as impact detection (KY-031), motion tracking (MPU6050), heart rate monitoring (MAX30102), and inactivity detection (PIR) on a walking cane, the device detects critical incidents like falls or health emergencies in real time. Equipped with a buzzer for immediate audible alerts, an LED for visual feedback, and an ESP32 module for wireless notifications to a designated Telegram chat, it enables remote oversight by caregivers or family members. The OLED display provides status updates, while an “I’m OK” button helps manage false alarms. This solution aims to offer freedom and security by keeping users connected to their support network, ensuring timely assistance without compromising their independence.

4.2 Objectives

- **Accurate Fall Detection:** Achieve precise identification of falls by analysing a combination of acceleration and orientation data from the MPU6050, ensuring reliable detection even in complex movement scenarios.
- **Real-Time Heart Rate Monitoring:** Continuously monitor heart rate using the MAX30102 sensor, triggering alerts for abnormal readings (outside 40-180 BPM) or when the sensor disconnects, indicating potential health risks.
- **Prolonged Inactivity Detection:** Utilise the PIR sensor to detect extended periods of immobility (beyond 4 seconds), activating alerts to address situations where the user may be incapacitated or unable to respond.
- **Minimise False Positives:** Implement a multi-trigger validation system that requires confirmation across multiple sensors (up to 4 triggers for CRITICO state) and include an “I’m OK” button to allow users to cancel false alarms, enhancing system accuracy and reducing unnecessary notifications.
- **Real-Time Alert Transmission:** Leverage Wi-Fi connectivity via the ESP32 to send immediate, detailed alerts to a Telegram chat, ensuring caregivers or family members are promptly informed of emergencies with specific trigger details.
- **Status and Vital Signs Display:** Present real-time system status, including the current state (e.g., NORMAL, CRITICAL), trigger count, and vital signs (e.g., heart rate) on an OLED screen, providing users and caregivers with clear, at-a-glance information.
- **Reliable, Low-Cost, Non-Intrusive Support:** Develop a robust, affordable device that integrates seamlessly into daily life as a walking aid, offering a non-invasive monitoring solution that supports caregivers and family members by delivering actionable insights.



AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

5.0 Requirements

The Smart Cane system shall meet the following requirements:

Functional Requirements:

- **Fall Detection:** Swiftly detect falls using the MPU6050 accelerometer and gyroscope, identifying incidents within moments to ensure immediate awareness.
- **Heart Rate Monitoring:** Track heart rate accurately with the MAX30102 sensor, providing reliable updates to flag potential health concerns as they arise.
- **Inactivity Detection:** Recognise prolonged inactivity using the PIR sensor, alerting caregivers to situations where the user may need assistance.
- **Telegram Alerts:** Send instant alerts to predefined contacts via Telegram, keeping caregivers informed of critical events like falls or health issues with minimal delay.
- **Real-Time Display:** Continuously update heart rate, system status, and alerts on a 0.96-inch OLED display, offering users and caregivers a clear, live view of the patient's condition.
- **Obstacle Awareness:** Detect nearby obstacles using an ultrasonic sensor, enhancing user safety by preventing collisions during movement.

• Performance Requirements:

- **Extended Operation:** Operate reliably for over 12 hours on a single battery charge, ensuring uninterrupted support throughout the day.
- **Continuous Updates:** Refresh sensor data and display information every 1 second, maintaining a seamless flow of real-time insights for both user and caregiver.

• Environmental Requirements:

- **Versatile Conditions:** Perform effectively in a wide range of temperatures and light rain, making it a dependable companion for indoor and outdoor use.
- **Durable Design:** Built to withstand everyday environmental challenges, ensuring consistent performance in varied settings.

• Safety Requirements:

- **User-Friendly Design:** Crafted with comfortable, safe materials for prolonged contact, prioritising the well-being of the user.
- **Secure Operation:** Engineered to provide a stable and safe experience, focusing on the user's peace of mind.

6.0 Project Schedule

AFSC: ANTI FALL SMART CANE

Prototipado
16 de Julio al 25 de Julio del 2025



7.0 Materials, Equipment & Total Cost

7.1 Microcontroller(s)

The ESP32-WROOM-32 module was selected for the Smart Cane due to its dual-core processor, integrated Wi-Fi/Bluetooth connectivity, and 38 GPIO pins, enabling efficient sensor integration (MPU6050, MAX30102, KY-031, PIR) and real-time communication. Operating at 3.3V with a clock speed of up to 240 MHz, it ensures rapid processing for fall detection and health monitoring. The ESP32 was chosen over the Arduino Uno for its built-in wireless capabilities (eliminating the need for extra



AFSC: ANTI FALL SMART CANE

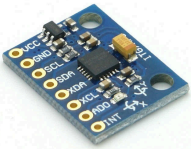
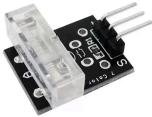
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

shields), higher processing power (240 MHz vs. 16 MHz), and lower cost (~\$5 vs. \$20), supporting continuous caregiver updates and the project's 12-hour battery life goal. Its extensive community support further enhances development flexibility, making it an ideal, cost-effective choice for reliable remote monitoring.

7.2 Sensors and Actuators

Table 1: Sensor Specifications and Characteristics

Sensor	Image	Function	Operating	Error
MPU6050		Fall detection (accelerometer, gyroscope)	3.3V	$\pm 3\%$
MAX30102		Heart rate monitoring	1.8V-3.3V	± 5 bpm
Mini PIR Sensor		Inactivity detection	3.3V-5V	N/A
KY-031		Impact detection	5V	± 3 mm



**AFSC: ANTI FALL
SMART CANE**
Critical Design Document

**IET400_CDR_V1.0
Revision 0.1**

The sensors selected for the Smart Cane—MPU6050, MAX30102, KY-031, and PIR HW-456-SR505—are ideal for this project due to their combination of accuracy, compatibility, and specific functionalities that align with the objectives of fall detection, health monitoring, and safety. The MPU6050, with its integrated accelerometer and gyroscope, offers fast and reliable fall detection by analysing real-time acceleration and orientation, which is essential in critical emergencies. The MAX30102 stands out for its ability to measure heart rate with high accuracy (± 5 bpm) using infrared light technology, enabling continuous monitoring and anomaly alerts, making it ideal for the health of vulnerable users. The KY-031, as an impact sensor, detects physical collisions with adjustable sensitivity, complementing fall detection, while the PIR HW-456-SR505 identifies prolonged inactivity with low power consumption, ensuring timely alerts without draining the battery. These sensors, integrated with the ESP32, fit perfectly into the portable and low-cost design of the cane, providing a robust, non-invasive, and efficient system to support users and caregivers effectively.

7.3 Other Components Required

Components	Image	Function	Operating	Error
Buzzer		Generate audible warning signals when an emergency is detected.	The piezoelectric buzzer works by vibrating a piezoelectric disc, which, when an electrical signal (often PWM) is applied, generates audible sound waves. It is activated by applying a DC voltage (usually 3.3V or 5V).	N/A

Pantalla oled		Display relevant system information such as patient status, heart rate (BPM), alerts, and status messages.	It uses OLED (organic light-emitting diode) technology to display text and icons. It does not require backlighting and communicates with the microcontroller (ESP32 or Arduino) via the I2C interface (SDA/SCL).	N/A
Barra led		Provide a visual warning signal with different flashing modes depending on the emergency level or number of events (triggers).	When voltage is applied to the LED anode through a limiting resistor, visible light is generated. The flashing or brightness can be controlled by code (digitalWrite or PWM).	N/A
Boton		Allow the user to cancel an alert in case of a false positive or to confirm that they are okay before an emergency is triggered.	It is a mechanical switch that, when pressed, closes a circuit by sending a digital signal (usually LOW or HIGH) to the microcontroller. It	N/A



**AFSC: ANTI FALL
SMART CANE**
Critical Design Document

**IET400_CDR_V1.0
Revision 0.1**

is used in
conjunction with a
pull-up or
pull-down resistor.

7.4 Total Cost of System

Table 2: Component Cost Breakdown

Component	Individual Cost (\$)	Quantity
ESP32-WROOM-32	125 MXN	1
MPU6050	75 MXN	1
MAX30102	170 MXN	1
Mini PIR Sensor	57 MXN	1
KY-031	61 MXN	3
OLED Display	126 MXN	1
Li-Po Battery 3.7v	150 MXN	1
CN6009	55 MXN	1
Buzzer	35 MXN	1
Cane Frame	250 MXN	1
Miscellaneous (wires, resistors)	250 MXN	N/A
TOTAL	1354 MXN	



AFSC: ANTI FALL SMART CANE

Critical Design Document

IET400_CDR_V1.0
Revision 0.1

8.0 System Design

ID#	Sensing Parameter	TYPE	Operating Voltage	Working Amperage	Temperature Range	Accuracy	Interface Type	Pins (other than VCC and GND)
MPU6050	Motion, Fall Detection	6-Axis Accelerometer/Gyroscope	2.3-3.6V	3.6mA	-40 to 85°C	±3%	Digital, I2C	SDA, SCL, INT
MAX30102	Heart Rate, SpO2	Optical Pulse Oximeter	1.8-3.3V	1.2mA	-40 to 85°C	±5 bpm	Digital, I2C	SDA, SCL, INT
PIR	Inactivity Detection	Passive Infrared	3.3-5V	0.1mA	-20 to 60°C	N/A	Digital	OUT, EN
KY-031	Impact Detection	Tilt/Shock Switch	3.3-5V	0.5mA	-10 to 70°C	N/A	Digital	DO

8.1 Structural-Mechanical System Design Diagram





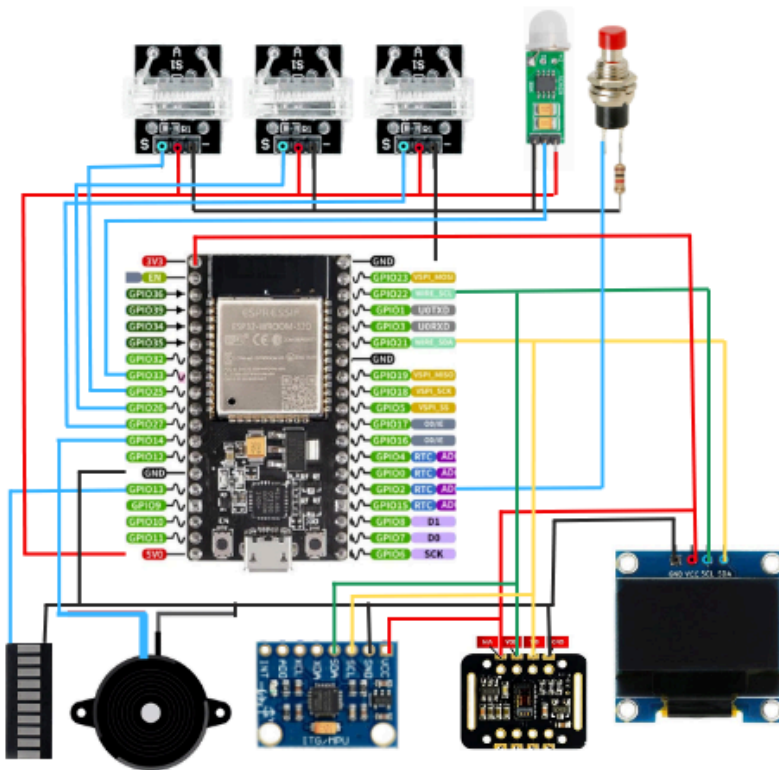
AFSC: ANTI FALL SMART CANE

Critical Design Document

IET400_CDR_V1.0
Revision 0.1



8.2 Electrical System Design Diagram





AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

8.3 Sensor System Code

```
#include <Wire.h>
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <UniversalTelegramBot.h>
#include <MPU6050_tockn.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <MAX30105.h>
#include <heartRate.h>
#include <esp_system.h>

// Pines
#define IMPACT_PIN_1 27 // KY-031
#define IMPACT_PIN_2 26 // KY-031
#define IMPACT_PIN_3 25 // KY-031
#define BUZZER_PIN 14 // Buzzer
#define BUTTON_PIN 2 // Botón (pull-up externo 4.7kΩ a 3.3V, botón a GND, HIGH al
presionar)
#define PIR_PIN 33 // Mini PIR HW-456-SR505
#define LED_PIN 13 // Barra LED
#define SDA_PIN 21
#define SCL_PIN 22

// Pantalla
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);

// Sensores
MPU6050 mpu(Wire);
MAX30105 particleSensor;

// Telegram
const char* ssid = "*****",
const char* password = "*****",
#define BOT_TOKEN "*****"
#define CHAT_ID "*****"
WiFiClientSecure client;
UniversalTelegramBot bot(BOT_TOKEN, client);
```



AFSC: ANTI FALL SMART CANE

Critical Design Document

IET400_CDR_V1.0
Revision 0.1

// Estructuras

```
struct SensorStatus {  
    bool impact = false;  
    bool mpu = false;  
    bool heart = false;  
    bool pir = false;  
    bool wifi = false;  
};  
  
struct Trigger {  
    bool active = false;  
    unsigned long time = 0;  
    bool latched = false; // Estado retenido temporal  
};
```

// Variables globales

```
SensorStatus sensors;  
volatile Trigger mpuTrigger = {false, 0, false};  
Trigger impactTrigger = {false, 0, false};  
Trigger heartTrigger = {false, 0, false};  
Trigger pirTrigger = {false, 0, false};  
int triggerCount = 0;  
int lastTriggerCount = 0;  
unsigned long lastNotificationTime = 0;  
int totalSensors = 0;  
bool threeTriggerNotified = false; // Flag para 3 triggers
```

// Estados

```
enum State { CALIBRATING, NORMAL, ALERT_1, ALERT_2, ALERT_3, CRITICO, CANCELLED };  
State currentState = CALIBRATING;
```

// MPU Parameters

```
const float FALL_ACCEL_THRESHOLD = 1.2;  
const float ANGLE_TOLERANCE = 25.0;  
const unsigned long CONFIRM_DELAY = 5000;  
const unsigned long MOTION_TIMEOUT = 40000; // 40 segundos  
const unsigned long CRITICO_TIMEOUT = 10000; // 10 segundos para presionar botón en estado CRITICO
```

// MAX30102 Parameters

```
const long IR_THRESHOLD = 50000;  
const int BPM_MIN = 40, BPM_MAX = 180;
```



AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

```
const byte RATE_SIZE = 4;

// PIR Parameters
const unsigned long PIR_WARMUP = 30000;
const unsigned long PIR_NO_MOTION_TIMEOUT = 4000; // 4 segundos

// Impacto Parameters
const int IMPACT_SAMPLES = 10;

// Variables MPU
float pitchCal = 0, rollCal = 0;
bool mpuCalibrated = false;
float lastAccel = 0;
unsigned long mpuAlertStartTime = 0;
enum MPUState { MPU_NORMAL, MPU_ALERT, MPU_CONFIRMED } mpuState = MPU_NORMAL;
unsigned long lastMpuUpdate = 0; // Para detectar bloqueos

// Variables BPM
byte rates[RATE_SIZE] = {0};
byte rateSpot = 0;
long lastBeat = 0;
int stableBeatAvg = 0;

// Variables de tiempo
unsigned long stateStartTime = 0;
unsigned long lastUpdate = 0;
unsigned long displayUpdate = 0;
unsigned long ledBlinkTime = 0;
unsigned long pirWarmupStart = 0;
bool ledState = false;

// Variables botón
bool buttonPressed = false;
unsigned long lastButtonTime = 0;
const unsigned long DEBOUNCE_TIME = 200;
int buttonReadCount = 0;

#define NOTIFICATION_COOLDOWN 60000

// Variables para mensajes temporales
String tempMessage = "";
unsigned long tempMessageStart = 0;
```



AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

```
const unsigned long TEMP_MESSAGE_DURATION = 500; // Duración para parpadeo
bool tempMessageBlink = false;
```

```
// Función para mostrar texto en OLED
```

```
void displayText(String text) {
    display.clearDisplay();
    display.setTextSize(1);
    display.setTextColor(WHITE);
    display.setCursor(0, 20);
    display.println(text);
    display.display();
}
```

```
void mostrarMensajeGrandeOLED(String mensaje) {
    tempMessage = mensaje;
    tempMessageStart = millis();
    tempMessageBlink = true;
}
```

```
void drawHeartIcon(int x, int y) {
    static const unsigned char heartBitmap[] PROGMEM = {
        0b00001100, 0b00110000,
        0b00011110, 0b01111000,
        0b00111111, 0b11111100,
        0b01111111, 0b11111110,
        0b01111111, 0b11111110,
        0b01111111, 0b11111110,
        0b01111111, 0b11111110,
        0b00111111, 0b11111100,
        0b00011111, 0b11111000,
        0b00001111, 0b11110000,
        0b00000111, 0b11100000,
        0b00000011, 0b11000000,
        0b00000001, 0b10000000,
        0b00000000, 0b00000000
    };
    display.drawBitmap(x, y, heartBitmap, 16, 13, WHITE);
}
```

```
// Setup
```

```
void setup() {
    Serial.begin(115200);
    Serial.println("=== Baston Inteligente V4.2.9 ===");
    Serial.print("Razon de reinicio: ");
}
```



AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

```
switch (esp_reset_reason()) {  
    case ESP_RST_POWERON: Serial.println("Power-on reset"); break;  
    case ESP_RST_EXT: Serial.println("External reset"); break;  
    case ESP_RST_SW: Serial.println("Software reset"); break;  
    case ESP_RST_PANIC: Serial.println("Exception/Panic"); break;  
    case ESP_RST_INT_WDT: Serial.println("Interrupt Watchdog"); break;  
    case ESP_RST_TASK_WDT: Serial.println("Task Watchdog"); break;  
    case ESP_RST_WDT: Serial.println("Other Watchdog"); break;  
    case ESP_RST_DEEPSLEEP: Serial.println("Deep Sleep reset"); break;  
    case ESP_RST_BROWNOUT: Serial.println("Brownout reset"); break;  
    default: Serial.println("Unknown"); break;  
}
```

Serial.println("Botón: Configurado en GPIO 2 con pull-up EXTERNO de 4.7kΩ a 3.3V (botón a GND, HIGH al presionar).");

```
pinMode(IMPACT_PIN_1, INPUT_PULLUP);  
pinMode(IMPACT_PIN_2, INPUT_PULLUP);  
pinMode(IMPACT_PIN_3, INPUT_PULLUP);  
pinMode(BUZZER_PIN, OUTPUT);  
pinMode(BUTTON_PIN, INPUT);  
pinMode(PIR_PIN, INPUT);  
pinMode(LED_PIN, OUTPUT);  
digitalWrite(BUZZER_PIN, LOW);  
digitalWrite(LED_PIN, LOW);
```

```
Wire.begin(SDA_PIN, SCL_PIN);  
Wire.setClock(100000);
```

```
byte oledAddress = scanI2C();  
bool oledOk = false;  
if (oledAddress != 0) {  
    if (display.begin(SSD1306_SWITCHCAPVCC, oledAddress)) {  
        oledOk = true;  
        Serial.println("OLED inicializado en 0x" + String(oledAddress, HEX));  
        displayText("Iniciando...");  
    }  
}  
if (!oledOk) {  
    Serial.println("Error: OLED no inicializado");  
}
```

```
checkSensors();  
connectWiFi();
```



AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

```
calibrateMPU();  
totalSensors = sensors.impact + sensors.mpu + sensors.heart + sensors.pir;  
Serial.println("Sensores activos: " + String(totalSensors));
```

```
if (sensors.pir) {  
    Serial.println("Calentando PIR HW-456-SR505...");  
    displayText("Calentando PIR...");  
    pirWarmupStart = millis();  
    while (millis() - pirWarmupStart < PIR_WARMUP) {  
        delay(100);  
    }  
}
```

```
if (oledOk) {  
    displayText("Iniciando...");  
    delay(2000);  
}  
currentState = NORMAL;  
}
```

```
// Escanear direcciones I2C  
byte scanI2C() {  
    Serial.println("Escaneando I2C...");  
    byte address = 0;  
    for (byte addr = 1; addr < 127; addr++) {  
        Wire.beginTransmission(addr);  
        if (Wire.endTransmission() == 0) {  
            Serial.println("Dispositivo en 0x" + String(addr, HEX));  
            if (addr == 0x3C || addr == 0x3D) {  
                address = addr;  
            } else if (addr == 0x57) {  
                sensors.heart = true;  
                Serial.println("MAX30102 detectado en 0x57");  
            }  
        }  
    }  
}  
if (address == 0) {  
    Serial.println("No se encontro OLED en I2C");  
}  
return address;  
}
```

```
// Verificación de sensores
```



AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

```
void checkSensors() {
    Serial.println("Verificando sensores...");
    int impactReadings[3] = { digitalRead(IMPACT_PIN_1), digitalRead(IMPACT_PIN_2),
digitalRead(IMPACT_PIN_3) };
    sensors.impact = (impactReadings[0] == HIGH && impactReadings[1] == HIGH &&
impactReadings[2] == HIGH);
    Serial.println("Impacto (KY-031): " + String(sensors.impact ? "OK" : "No detectado"));
    Serial.println(" Pin 27: " + String(impactReadings[0]));
    Serial.println(" Pin 26: " + String(impactReadings[1]));
    Serial.println(" Pin 25: " + String(impactReadings[2]));

    mpu.begin();
    Wire.beginTransmission(0x68);
    Wire.write(0x75);
    Wire.endTransmission(false);
    Wire.requestFrom(0x68, 1);
    sensors.mpu = (Wire.available() && Wire.read() == 0x68);
    Serial.println("MPU6050: " + String(sensors.mpu ? "OK" : "No detectado"));

    if (sensors.heart && particleSensor.begin(Wire, I2C_SPEED_FAST)) {
        particleSensor.setup();
        particleSensor.setPulseAmplitudeRed(0x0A);
        particleSensor.setPulseAmplitudeGreen(0);
        Serial.println("MAX30102: OK");
    } else {
        sensors.heart = false;
        Serial.println("MAX30102: No detectado");
    }

    sensors.pir = true;
    Serial.println("PIR (HW-456-SR505): OK");
    Serial.println(" PIR Pin 33: " + String(digitalRead(PIR_PIN)));
}

// Conexión WiFi
void connectWiFi() {
    Serial.print("Conectando WiFi...");
    WiFi.begin(ssid, password);
    int attempts = 0;
    while (WiFi.status() != WL_CONNECTED && attempts++ < 20) {
        delay(500);
        Serial.print(".");
    }
}
```



AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

```
sensors.wifi = WiFi.status() == WL_CONNECTED;
Serial.println(sensors.wifi ? "OK" : "Fallo");
if (sensors.wifi) {
    client.setInsecure();
    bot.sendMessage(CHAT_ID, "SMART CANE INIT", "");
}
displayText(sensors.wifi ? "WiFi OK" : "WiFi fallo");
delay(2000);
}

// Calibración MPU
void calibrateMPU() {
    if (!sensors.mpu) return;
    displayText("Calibrando MPU...");
    Serial.println("Calibrando MPU...");
    float pitchSum = 0, rollSum = 0;
    for (int i = 0; i < 20; i++) {
        mpu.update();
        pitchSum += mpu.getAngleX();
        rollSum += mpu.getAngleY();
        delay(100);
    }
    pitchCal = pitchSum / 20;
    rollCal = rollSum / 20;
    mpuCalibrated = true;
    Serial.println("MPU calibrado: PitchCal=" + String(pitchCal, 2) + ", RollCal=" + String(rollCal, 2));
}

// Loop principal
void loop() {
    unsigned long currentTime = millis();
    readSensors();
    checkButton();
    updateTriggers(currentTime);
    handleState(currentTime);
    updateDisplay();
    updateLED(currentTime);
    if (currentTime - lastUpdate > 2000) {
        printDebugInfo();
        lastUpdate = currentTime;
    }
    delay(20);
}
```




AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

```
// Lectura de sensores
void readSensors() {
    if (sensors.impact) readImpact();
    if (sensors.mpu && mpuCalibrated) readMPU();
    if (sensors.heart) readHeart();
    if (sensors.pir && millis() - pirWarmupStart > PIR_WARMUP) readPIR();
}

void readImpact() {
    int readings[3] = { HIGH, HIGH, HIGH };
    for (int i = 0; i < IMPACT_SAMPLES; i++) {
        readings[0] &= digitalRead(IMPACT_PIN_1);
        readings[1] &= digitalRead(IMPACT_PIN_2);
        readings[2] &= digitalRead(IMPACT_PIN_3);
        delay(2);
    }
    if ((readings[0] == LOW || readings[1] == LOW || readings[2] == LOW) && !impactTrigger.latched)
    {
        impactTrigger.active = true;
        impactTrigger.time = millis();
        impactTrigger.latched = true; // Activar retención
        Serial.println("Impacto detectado (KY-031) - Latch activado");
        sendTelegramNotification();
    }
    // Ajuste a 60 segundos para impacto
    if (impactTrigger.latched && (millis() - impactTrigger.time > 60000)) {
        impactTrigger.active = false;
        impactTrigger.latched = false;
        Serial.println("Impacto - Timeout alcanzado (60s), latch desactivado");
    }
}

void readMPU() {
    mpu.update();
    float accX = mpu.getAccX();
    float accY = mpu.getAccY();
    float accZ = mpu.getAccZ();
    float currentAccel = sqrt(accX * accX + accY * accY + accZ * accZ);
    float deltaAccel = abs(currentAccel - lastAccel);
    lastAccel = currentAccel;
    float pitch = mpu.getAngleX();
    float roll = mpu.getAngleY();
}
```



AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

```
// Detectar y reiniciar si el MPU está bloqueado
if (accX == 0.00 && accY == 0.00 && accZ == 0.00 && (millis() - lastMpuUpdate > 1000)) {
    Serial.println("MPU bloqueado detectado, reiniciando...");
    mpu.begin();
    mpuCalibrated = false;
    calibrateMPU();
    return;
}
lastMpuUpdate = millis();

Serial.println("MPU - Raw Acc: " + String(accX, 2) + ", " + String(accY, 2) + ", " + String(accZ, 2) +
" Delta: " + String(deltaAccel, 2));
if (deltaAccel > FALL_ACCEL_THRESHOLD && mpuState == MPU_NORMAL &&
!mpuTrigger.latched) {
    Serial.println("MPU - Condición de caída detectada (subtrigger)");
    mpuState = MPU_ALERT;
    mpuAlertStartTime = millis();
    mpuTrigger.latched = true; // Activar estado retenido
    mpuTrigger.time = millis();
    digitalWrite(BUZZER_PIN, HIGH);
    mostrarMensajeGrandeOLED("CAÍDA");
} else if (mpuState == MPU_ALERT && millis() - mpuAlertStartTime >= CONFIRM_DELAY) {
    Serial.println("MPU - Evaluando confirmación");
    float pitchDiff = abs(pitch - pitchCal);
    float rollDiff = abs(roll - rollCal);
    Serial.println("MPU - PitchDiff: " + String(pitchDiff, 2) + " RollDiff: " + String(rollDiff, 2));
    if ((pitchDiff > ANGLE_TOLERANCE || rollDiff > ANGLE_TOLERANCE)) {
        mpuState = MPU_CONFIRMED;
        mpuTrigger.active = true; // Activar trigger general tras confirmación
        mpuTrigger.latched = false; // Resolver el estado retenido
        mostrarMensajeGrandeOLED("CONFIRMADA");
        Serial.println("MPU - Caída confirmada. PitchDiff: " + String(pitchDiff, 2) + " RollDiff: " +
String(rollDiff, 2) + " Trigger: " + String(mpuTrigger.active));
    } else {
        mpuState = MPU_NORMAL; // Si no se confirma, volver a normal
        mpuTrigger.latched = false; // Resolver el estado retenido
        digitalWrite(BUZZER_PIN, LOW);
    }
}
// Mantener trigger activo durante 40 segundos tras confirmación
if (mpuTrigger.active && (millis() - mpuTrigger.time > MOTION_TIMEOUT)) {
    mpuState = MPU_NORMAL;
```



AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

```
mpuTrigger.active = false;
digitalWrite(BUZZER_PIN, LOW);
Serial.println("MPU - Regresó a normal (40s). Trigger: " + String(mpuTrigger.active));
}
}

void readHeart() {
    long irValue = particleSensor.getIR();
    Serial.println("Heart - IR: " + String(irValue) + " Beat: " + String(checkForBeat(irValue)));
    if (irValue < 5000) {
        for (byte x = 0; x < RATE_SIZE; x++) rates[x] = 0;
        stableBeatAvg = 0;
        heartTrigger.active = true;
        heartTrigger.time = millis();
        heartTrigger.latched = true; // Activar retención
        Serial.println("Sin señal en MAX30102 (IR=" + String(irValue) + ")");
        sendTelegramNotification();
    } else if (checkForBeat(irValue) && irValue > 5000) {
        long delta = millis() - lastBeat;
        lastBeat = millis();
        if (delta > 250) {
            float beatsPerMinute = 60000 / delta;
            if (beatsPerMinute >= BPM_MIN && beatsPerMinute <= BPM_MAX) {
                rates[rateSpot++] = (byte)beatsPerMinute;
                rateSpot %= RATE_SIZE;
                stableBeatAvg = 0;
                for (byte x = 0; x < RATE_SIZE; x++) {
                    stableBeatAvg += rates[x];
                }
                stableBeatAvg /= RATE_SIZE;
                Serial.println("Heart - New BPM: " + String(stableBeatAvg));
            }
        }
    }
    } else if (irValue > IR_THRESHOLD && heartTrigger.latched) {
        heartTrigger.active = false;
        heartTrigger.latched = false;
    }
}

void readPIR() {
    static unsigned long lastMotion = 0;
    int pirState = digitalRead(PIR_PIN);
    if (pirState == HIGH) {
```



AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

```
    lastMotion = millis();
    if (pirTrigger.latched) {
        pirTrigger.active = false;
        pirTrigger.latched = false;
        Serial.println("Movimiento detectado (HW-456-SR505) - Latch desactivado");
    }
} else if (millis() - lastMotion > PIR_NO_MOTION_TIMEOUT && !pirTrigger.latched) {
    pirTrigger.active = true;
    pirTrigger.time = millis();
    pirTrigger.latched = true; // Activar retención
    Serial.println("Sin movimiento por 4s (HW-456-SR505) - Latch activado");
    sendTelegramNotification();
}
}

// Verificación del botón
void checkButton() {
    int v_state = digitalRead(BUTTON_PIN);
    buttonReadCount++;
    Serial.println("Botón (GPIO 2) lectura #" + String(buttonReadCount) + ": Pin=" + String(v_state));

    if (v_state == HIGH && millis() - lastButtonTime > DEBOUNCE_TIME) {
        buttonPressed = true;
        Serial.println("PULSADO en estado " + getStateName(currentState));
        lastButtonTime = millis();
    }
    delay(10);
}

// Enviar notificación de Telegram
void sendTelegramNotification() {
    if (!sensors.wifi) {
        Serial.println("No hay conexión WiFi, intentando reconectar...");
        connectWiFi();
        if (!sensors.wifi) {
            Serial.println("Reconexión WiFi fallida, notificación no enviada");
            return;
        }
    }
}

if (triggerCount >= 2 && lastTriggerCount < 2) {
    String message = "● ALERTA NIVEL 1 POSIBLE CAIDA DETECTADA\nTriggers: " +
String(triggerCount);
    if (sensors.impact && impactTrigger.active) message += "\n- Impacto";
}
```



AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

```
if (sensors.mpu && mpuTrigger.active) message += "\n- Caída";
if (sensors.heart && heartTrigger.active) message += "\n- BPM: Sin señal";
if (sensors.pir && pirTrigger.active) message += "\n- Sin movimiento";
bool sent = bot.sendMessage(CHAT_ID, message, "");
Serial.println("Notificación enviada: " + message + " - " + String(sent ? "Éxito" : "Fallo"));
lastNotificationTime = millis();
}
// Notificación inmediata para 3 triggers
if (triggerCount == 3 && !threeTriggerNotified) {
  if (sensors.wifi) {
    bool sent = bot.sendMessage(CHAT_ID, "🟡 ALERTA NIVEL 2\nTriggers: 3", "");
    Serial.println("Notificación 3 triggers: " + String(sent ? "Éxito" : "Fallo"));
    threeTriggerNotified = true; // Marcar como notificado
    lastNotificationTime = millis(); // Actualizar cooldown después de enviar
  }
}
// Resetear flag cuando triggers bajen de 3
if (triggerCount < 3) {
  threeTriggerNotified = false;
}
// Serie de 5 mensajes para 4 triggers
if (triggerCount == 4) {
  Serial.println("Entrando en modo de 5 alertas críticas por 4 triggers");
  if (sensors.wifi) {
    for (int i = 0; i < 5; i++) {
      Serial.print("Intentando enviar alerta crítica #" + String(i + 1) + "... ");
      bool sent = bot.sendMessage(CHAT_ID, "🔴 ALERTA CRITICA CAIDA CONFIRMADA
PONGASE EN CONTACTO CON SU PACIENTE\nTriggers: 4", "");
      Serial.println(sent ? "Éxito" : "Fallo");
      if (!sent) {
        Serial.println("Fallo en el intento #" + String(i + 1) + ", intentando reconectar...");
        connectWiFi();
        if (sensors.wifi) {
          sent = bot.sendMessage(CHAT_ID, "🔴 ALERTA CRITICA CAIDA CONFIRMADA
PONGASE EN CONTACTO CON SU PACIENTE \nTriggers: 4", "");
          Serial.println(sent ? "Reintento exitoso" : "Reintento fallido");
        }
      }
    }
    delay(300); // Aumentar retraso para estabilidad
  }
  lastNotificationTime = millis(); // Respetar cooldown después de la serie
} else {
  Serial.println("WiFi no disponible para enviar alertas críticas");
}
```



AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

```
}  
}  
}
```

// Actualización de triggers

```
void updateTriggers(unsigned long currentTime) {  
    int prevTriggerCount = triggerCount;  
    triggerCount = (sensors.impact && impactTrigger.active) +  
        (sensors.mpu && mpuTrigger.active) +  
        (sensors.heart && heartTrigger.active) +  
        (sensors.pir && pirTrigger.active);  
  
    if (triggerCount >= 2 && prevTriggerCount < 2) {  
        sendTelegramNotification();  
    }  
    lastTriggerCount = triggerCount;  
}
```

// Manejo de estados

```
void handleState(unsigned long currentTime) {  
    State newState = NORMAL;  
    if (triggerCount == 1) newState = ALERT_1;  
    else if (triggerCount == 2) newState = ALERT_2;  
    else if (triggerCount == 3) newState = ALERT_3;  
    else if (triggerCount == 4) newState = CRITICO;  
  
    if (newState != currentState) {  
        stateStartTime = currentTime;  
        currentState = newState;  
        Serial.println("Estado cambiado a: " + getStateName(currentState));  
        if (currentState == ALERT_1) {  
            digitalWrite(BUZZER_PIN, HIGH);  
            delay(100);  
            digitalWrite(BUZZER_PIN, LOW);  
        } else if (currentState == ALERT_2) {  
            digitalWrite(BUZZER_PIN, HIGH);  
            delay(100);  
            digitalWrite(BUZZER_PIN, LOW);  
            delay(100);  
            digitalWrite(BUZZER_PIN, HIGH);  
            delay(100);  
            digitalWrite(BUZZER_PIN, LOW);  
        } else if (currentState == ALERT_3) {
```



AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

```
digitalWrite(BUZZER_PIN, HIGH);
delay(300);
digitalWrite(BUZZER_PIN, LOW);
} else if (currentState == CRITICO) {
    digitalWrite(BUZZER_PIN, HIGH);
    mostrarMensajeGrandeOLED("CRITICO");
}
}

switch (currentState) {
    case ALERT_1:
    case ALERT_2:
        if (currentTime - stateStartTime > MOTION_TIMEOUT) {
            resetTriggers();
            currentState = NORMAL;
        }
        break;
    case ALERT_3:
        if (buttonPressed) {
            resetTriggers();
            currentState = CANCELLED;
            if (sensors.wifi) bot.sendMessage(CHAT_ID, "● Paciente oprimió botón de seguridad, falsa alarma, pongase en contacto con su paciente para confirmar su seguridad", "");
        }
        break;
    case CRITICO:
        if (buttonPressed) {
            Serial.println("Botón presionado en CRITICO, enviando notificación");
            resetTriggers();
            currentState = CANCELLED;
            if (sensors.wifi) bot.sendMessage(CHAT_ID, "● Paciente oprimió botón de seguridad, falsa alarma, pongase en contacto con su paciente para confirmar su seguridad", "");
        } else if (currentTime - stateStartTime > CRITICO_TIMEOUT) {
            Serial.println("Timeout de 10s alcanzado en CRITICO, enviando notificaciones");
            sendTelegramNotification();
            resetTriggers();
            currentState = NORMAL;
        }
        break;
    case CANCELLED:
        if (currentTime - stateStartTime > 3000) {
            currentState = NORMAL;
            digitalWrite(BUZZER_PIN, LOW);
        }
}
```



AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

```
}
    break;
}
buttonPressed = false;
}

void resetTriggers() {
    impactTrigger.active = false;
    impactTrigger.latched = false;
    mpuTrigger.active = false;
    mpuTrigger.latched = false;
    heartTrigger.active = false;
    heartTrigger.latched = false;
    pirTrigger.active = false;
    pirTrigger.latched = false;
    triggerCount = 0;
    lastTriggerCount = 0;
    mpuState = MPU_NORMAL;
    threeTriggerNotified = false; // Resetea flag al cancelar
    digitalWrite(BUZZER_PIN, LOW);
}

// Actualización LED
void updateLED(unsigned long currentTime) {
    unsigned long interval;
    switch (triggerCount) {
        case 0: digitalWrite(LED_PIN, LOW); return;
        case 1: interval = 1000; break;
        case 2: interval = 500; break;
        case 3: interval = 250; break;
        case 4: interval = 125; break; // Añadido para 4 triggers
        default: digitalWrite(LED_PIN, HIGH); return;
    }
    if (currentTime - ledBlinkTime > interval) {
        ledState = !ledState;
        digitalWrite(LED_PIN, ledState);
        ledBlinkTime = currentTime;
    }
}

// Actualización pantalla OLED
void updateDisplay() {
    if (millis() - displayUpdate < 100) return; // Aumentar frecuencia de actualización
```




AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

```
displayUpdate = millis();
```

```
if (tempMessage != "" && (millis() - tempMessageStart < TEMP_MESSAGE_DURATION * 10)) {  
// Duración extendida a 5 segundos  
    display.clearDisplay();  
    display.setTextSize(2);  
    int16_t x1, y1;  
    uint16_t w, h;  
    display.getTextBounds(tempMessage, 0, 0, &x1, &y1, &w, &h);  
    int16_t x = (SCREEN_WIDTH - w) / 2;  
    int16_t y = (SCREEN_HEIGHT - h) / 2;  
    if (tempMessageBlink && ((millis() / 500) % 2 == 0)) { // Parpadeo cada 500 ms  
        display.setTextColor(WHITE);  
        display.setCursor(x, y);  
        display.println(tempMessage);  
    } else if (tempMessageBlink) {  
        display.setTextColor(BLACK); // Apagar texto durante parpadeo  
        display.setCursor(x, y);  
        display.println(tempMessage);  
    }  
    display.display();  
    return;  
}
```

```
display.clearDisplay();  
display.setTextSize(1);  
display.setTextColor(WHITE);
```

```
display.setCursor(0, 0);  
display.print("Estado: ");  
display.println(getStateName(currentState));
```

```
display.setCursor(0, 15);  
display.print("Triggers: ");  
display.println(String(triggerCount) + "/" + String(totalSensors));
```

```
display.setCursor(0, 25);  
display.print("Impacto: ");  
display.println(sensors.impact ? (impactTrigger.active ? "Activo" : "OK") : "N/A");
```

```
display.setCursor(0, 35);  
display.print("MPU: ");  
display.println(sensors.mpu ? (mpuTrigger.active ? "Caída" : "OK") : "N/A");
```



AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

```
display.setCursor(0, 45);
if (sensors.heart && stableBeatAvg > 0) {
    drawHeartIcon(0, 45);
    display.setCursor(20, 45);
    display.print(stableBeatAvg);
    display.print(" BPM");
} else {
    display.print("BPM: ");
    display.println(sensors.heart && heartTrigger.active ? "Sin señal" : "N/A");
}

display.setCursor(0, 55);
display.print("PIR: ");
display.println(sensors.pir ? (pirTrigger.active ? "Sin mov" : "Mov") : "N/A");

display.display();
}

String getStateName(State state) {
    switch (state) {
        case CALIBRATING: return "Calibrando";
        case NORMAL: return "Normal";
        case ALERT_1: return "Alerta 1";
        case ALERT_2: return "Alerta 2";
        case ALERT_3: return "Alerta 3";
        case CRITICO: return "Critico";
        case CANCELLED: return "Cancelado";
        default: return "Desconocido";
    }
}

void printDebugInfo() {
    Serial.println("--- Estado del Sistema ---");
    Serial.println("Estado: " + getStateName(currentState));
    Serial.println("Triggers: " + String(triggerCount) + "/" + String(totalSensors));
    Serial.println("Impacto: " + String(sensors.impact ? (impactTrigger.active ? "ACTIVO" : "OK") :
"N/A") + " Latch: " + String(impactTrigger.latched));
    if (sensors.mpu) {
        String mpuStatus = mpuTrigger.active ? "CAIDA" : "OK";
        Serial.println("MPU: " + mpuStatus + " State: " + String(mpuState) + " Trigger: " +
String(mpuTrigger.active) + " Latch: " + String(mpuTrigger.latched));
    } else {
```



AFSC: ANTI FALL SMART CANE

Critical Design Document

IET400_CDR_V1.0
Revision 0.1

```
Serial.println("MPU: N/A");  
}  
Serial.println("BPM: " + String(sensors.heart ? (stableBeatAvg > 0 ? String(stableBeatAvg) + "  
BPM" : "SIN SEÑAL") : "N/A") + " Latch: " + String(heartTrigger.latched));  
Serial.println("PIR: " + String(sensors.pir ? (pirTrigger.active ? "SIN MOV" : "MOV") : "N/A") + "  
Latch: " + String(pirTrigger.latched));  
Serial.println("WiFi: " + String(sensors.wifi ? "Conectado" : "Desconectado"));  
}
```

9.0 Test Plan

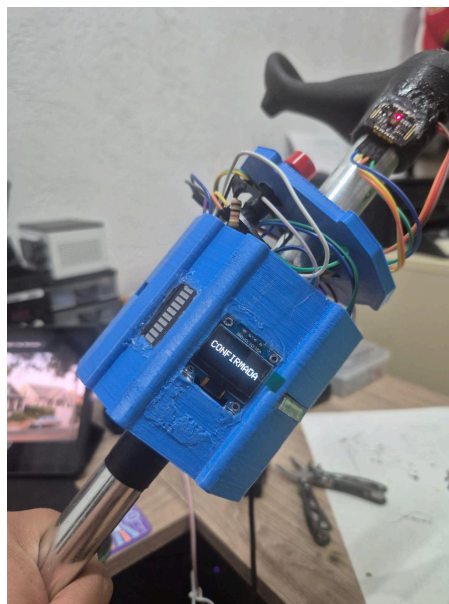
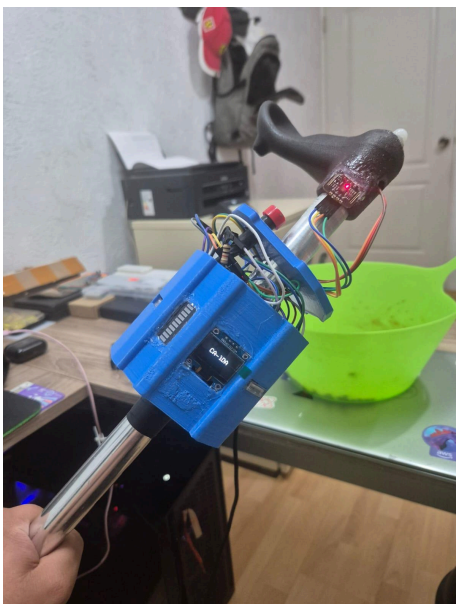
This test plan will be executed during the Critical Design Review (CDR) to validate the Smart Cane design and ensure its readiness for implementation. It provides a comprehensive assessment of system functionality, performance, and compatibility, addressing both electrical and mechanical aspects to guarantee user reliability and safety.

1. Preliminary Test Plan

The preliminary test plan focuses on individual component verification to ensure correct functionality before system integration.

Fall Detection (MPU6050)

- Test the accuracy of acceleration and orientation data.
- Verify detection of simulated falls within 2 seconds.





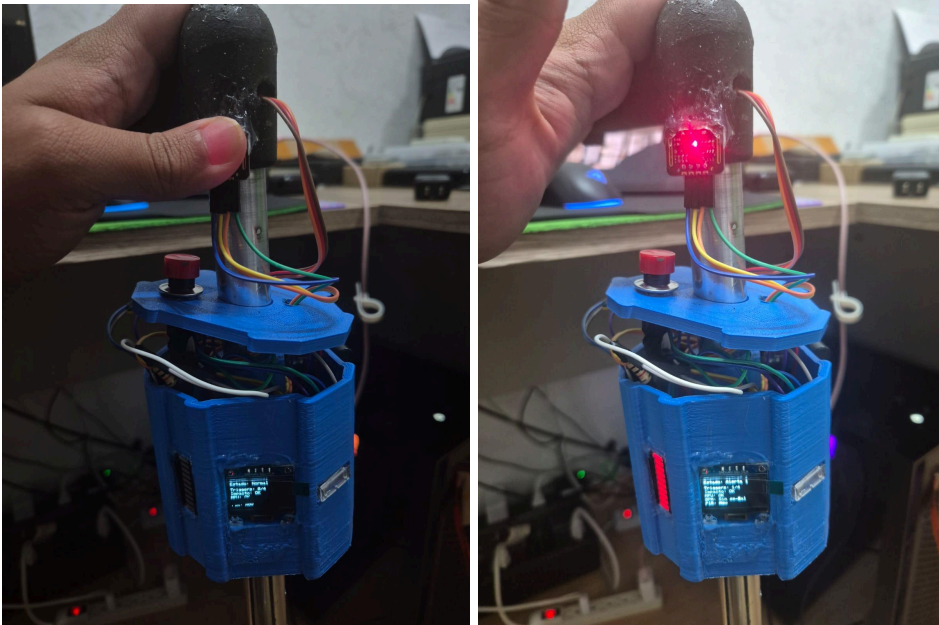
AFSC: ANTI FALL SMART CANE

Critical Design Document

IET400_CDR_V1.0
Revision 0.1

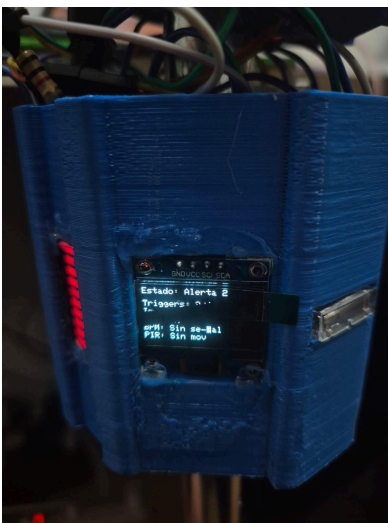
Heart Rate Monitoring (MAX30102)

- Measure heart rate accuracy (± 5 bpm).
- Test disconnection detection and alert activation.



Inactivity Detection (PIR)

- Simulate inactivity exceeding 5 minutes to trigger alerts.





AFSC: ANTI FALL SMART CANE

Critical Design Document

IET400_CDR_V1.0
Revision 0.1

Impact Detection (KY-031)

- Evaluate sensitivity and response time to impacts.

OLED Display

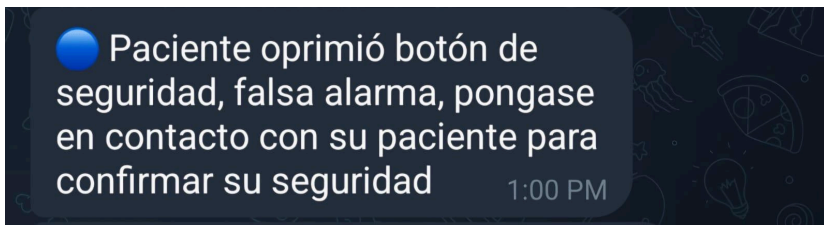
- Confirm display of heart rate, system state, and alerts.

Buzzer and LED

- Test audible and visual alerts for different activation levels.

“I’m OK” Button

- Verify functionality to cancel false alarms.



2. Integration and Test Plan

This stage ensures seamless interaction between components and verifies that system performance meets design requirements.

Electrical Functionality and Performance

- **Battery Testing:** Perform charge/discharge cycles to confirm 12-hour operation.
- **Voltage Measurement:** Ensure correct output from the power system (EPS).
- **EMI Testing:** Evaluate resistance to electromagnetic interference.

Software Functionality Tests

- Verify real-time data processing and 1-second screen update intervals.
- Test transitions in the state machine (NORMAL, ALERT_1, etc.).



AFSC: ANTI FALL SMART CANE

Critical Design Document

IET400_CDR_V1.0
Revision 0.1

- Confirm alert transmission via Telegram within 10 seconds of an event.

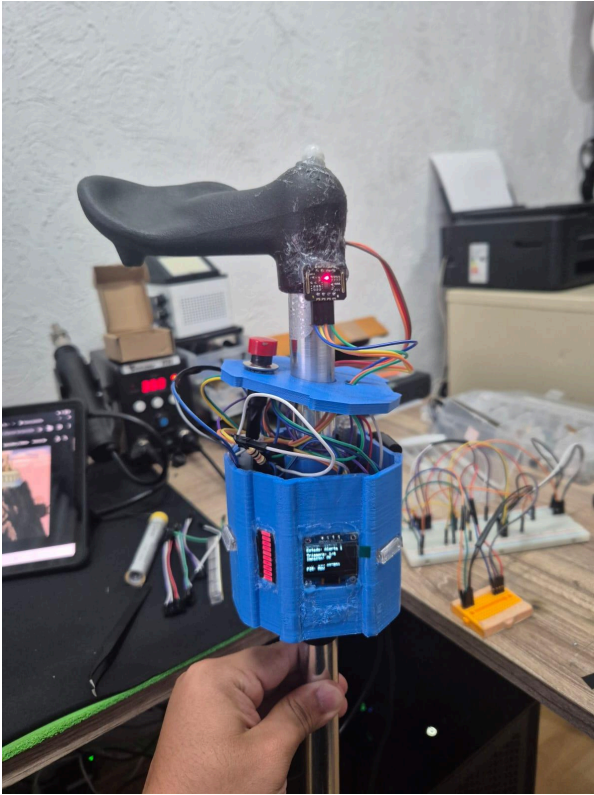
```
=== Baston Inteligente V4.2.9 ===  
Razon de reinicio: Power-on reset  
Botón: Configurado en GPIO 2 con pull-up EXTERNO de 4.7kΩ a 3.3V (botón a GND, HIGH al presionar).  
Escaneando I2C...  
Dispositivo en 0x3c  
Dispositivo en 0x57  
MAX30102 detectado en 0x57  
Dispositivo en 0x68  
OLED inicializado en 0x3c  
Verificando sensores...  
Impacto (KY-031): OK  
  Pin 27: 1  
  Pin 26: 1  
  Pin 25: 1  
MPU6050: OK  
MAX30102: OK  
PIR (HW-456-SR505): OK  
  PIR Pin 33: 0  
Conectando WiFi.....OK  
Calibrando MPU...  
MPU calibrado: PitchCal=-15.11, RollCal=95.91  
Sensores activos: 4  
Calentando PIR HW-456-SR505...  
MPU - Raw Acc: -0.94, -0.05, -0.11 Delta: 0.94  
Heart - IR: 14728 Beat: 0  
Sin movimiento por 4s (HW-456-SR505) - Latch activado  
Botón (GPIO 2) lectura #1: Pin=0  
Estado cambiado a: Alerta 1  
--- Estado del Sistema ---  
Estado: Alerta 1  
Triggers: 1/4  
Impacto: OK Latch: 0  
MPU: OK State: 0 Trigger: 0 Latch: 0  
BPM: SIN SEÑAL Latch: 0  
PIR: SIN MOV Latch: 1  
WiFi: Conectado  
MPU - Raw Acc: -0.94, -0.05, -0.13 Delta: 0.00
```




AFSC: ANTI FALL SMART CANE

Critical Design Document

IET400_CDR_V1.0
Revision 0.1



Mechanical Integration

- **Fit Checks:** Ensure all components fit within the cane's housing.
- **Cable Routing:** Verify cables are organized and do not interfere with components.
- **Assembly/Disassembly:** Assess ease of maintenance and repair.



3. System Validation Process

This phase validates that the Smart Cane meets user needs and operates reliably under real-world conditions.

Sensor Range Testing

- Assess detection range of all sensors (e.g., PIR inactivity, BPM).

Repeatability Testing

- Conduct multiple test cycles to ensure consistent performance.

Response Time Testing

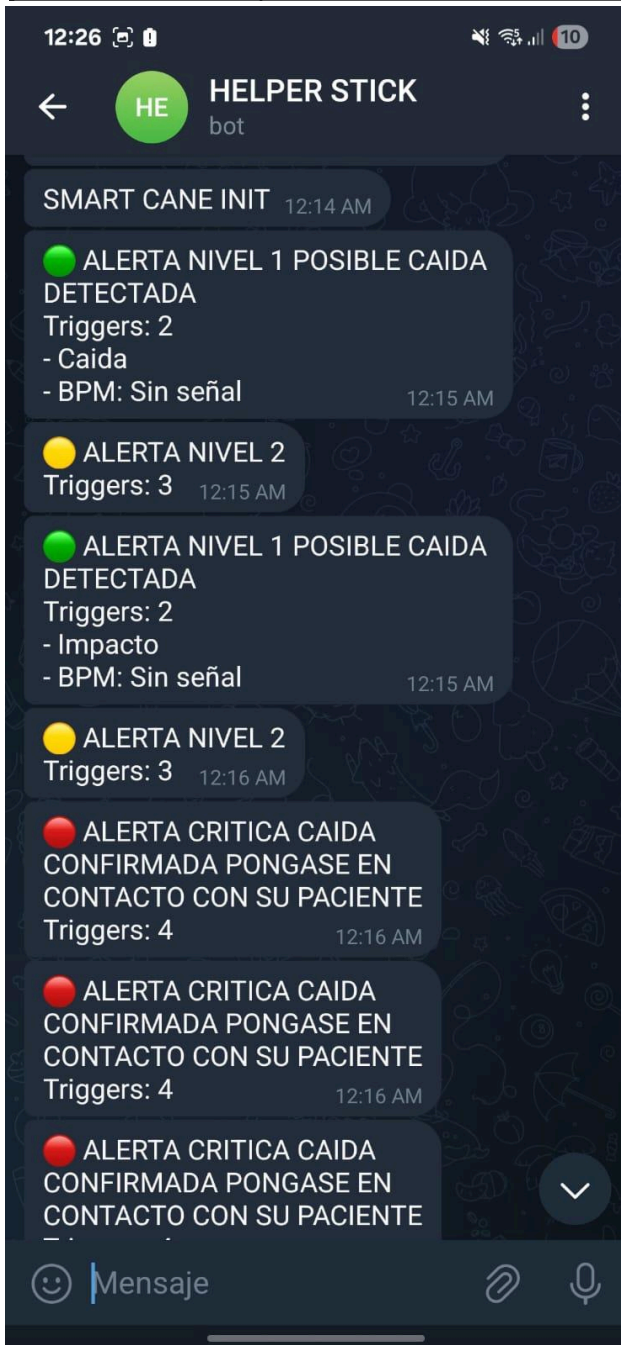
- Measure delay between event detection and alert transmission.



AFSC: ANTI FALL SMART CANE

Critical Design Document

IET400_CDR_V1.0
Revision 0.1



Environmental Testing

- Operate the cane in temperatures ranging from 0°C to 40°C.

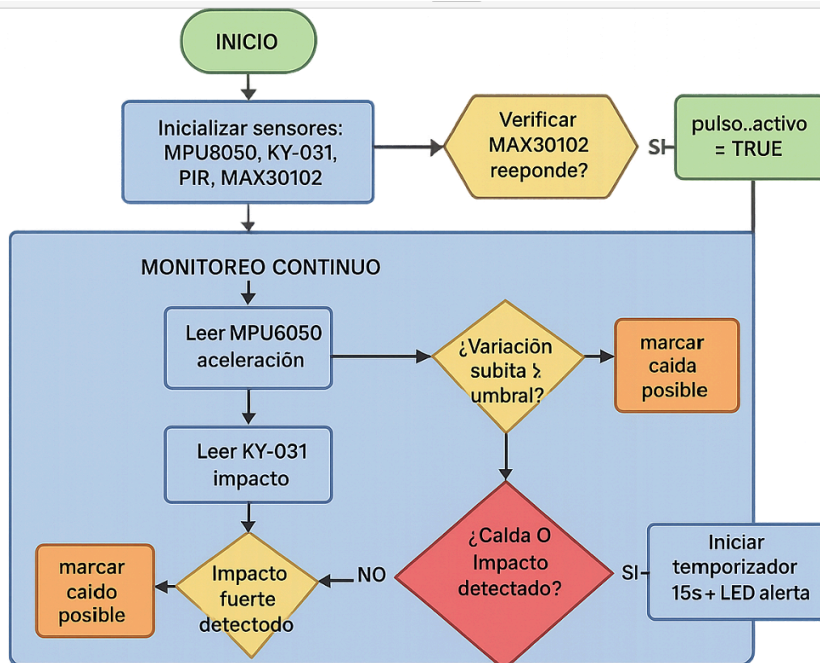
- Test under light rain conditions (IP54 rating).

Safety Testing

- Inspect for absence of exposed electrical components.
- Confirm biocompatibility of surfaces in contact with the user.

User-Centered Testing

- Simulate real-life usage scenarios (e.g., walking, falls).
- Test the effectiveness of the “I’m OK” button to cancel false alarms.



10.0 Glossary



AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

Acronym

Meaning

AT	<i>Assembly Test - A test conducted to verify the correct assembly of the Smart Cane's hardware components before integration.</i>
BPM	<i>Beats Per Minute - A unit measuring heart rate frequency, monitored by the MAX30102 sensor to detect health anomalies.</i>
CDR	<i>Critical Design Review - A formal evaluation to assess the Smart Cane's design maturity and readiness for production.</i>
CoG	<i>Center of Gravity - The point where the Smart Cane's weight is balanced, critical for stability during fall detection design.</i>
CPT	<i>Comprehensive Performance Test - A thorough evaluation of the Smart Cane's overall functionality under various conditions.</i>
EMI	<i>Electromagnetic Interference - External electrical noise that could affect the Smart Cane's sensor accuracy, mitigated by design.</i>
EMC	<i>Electromagnetic Compatibility - The ability of the Smart Cane to operate without causing or suffering from electromagnetic disturbances.</i>
ESP32	<i>Espressif Systems 32 - The microcontroller (ESP32-WROOM-32) powering the Smart Cane, known for Wi-Fi, dual-core processing, and GPIO flexibility.</i>
FIT	<i>Final Integrated Test - The ultimate test to confirm the Smart Cane's full integration and performance before deployment.</i>
FT	<i>Functional Test - A test to ensure each Smart Cane feature (e.g., fall detection, alerts) works as intended.</i>
GPIO	<i>General Purpose Input/Output - Configurable pins on the ESP32 used to connect sensors and actuators in the Smart Cane.</i>
I2C	<i>Inter-Integrated Circuit - A serial communication protocol used by the ESP32 to interface with the OLED and MPU6050.</i>
ITT	<i>Integrated Test - A test phase where Smart Cane subsystems (sensors, display, communication) are combined and evaluated.</i>



AFSC: ANTI FALL
SMART CANE
Critical Design Document

IET400_CDR_V1.0
Revision 0.1

LED	<i>Light Emitting Diode - A visual indicator on the Smart Cane that blinks to reflect trigger status.</i>
LPT	<i>Limited Performance Test - A partial evaluation of the Smart Cane's performance under specific conditions.</i>
PIR	<i>Passive Infrared - The HW-456-SR505 sensor detecting motion or inactivity in the Smart Cane.</i>
PIT	<i>Pre-integrated Test - A preliminary test of Smart Cane components before full system integration.</i>
PT	<i>Post Test - An evaluation conducted after deployment or modifications to assess the Smart Cane's ongoing performance.</i>
PDR	<i>Preliminary Design Review - An initial assessment of the Smart Cane's conceptual design and feasibility.</i>
UPAEP	<i>Universidad Popular Autónoma del Estado de Puebla - The academic institution potentially involved in the Smart Cane project's development.</i>
Wi-Fi	<i>Wireless Fidelity - The wireless networking technology in the ESP32, enabling Telegram alerts in the Smart Cane.</i>

11.0 Appendices and/or Annexes

- CIRCUIT FUNCTIONALITY: <https://youtu.be/Oy1licVQWoQ>